Aalto University

OPEN ACCESS

This is an electronic reprint of the original article.
This reprint may differ from the original in pagination and typographic detail.

King, Akira; Vyatkin, Valeriy

# LLM-based Iterative Refinement of Finite-State Machines with STPA Controller Constraints and Generation of IEC 61499 Code

*Published in:*
2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)

*DOI:*
10.1109/ETFA65518.2025.11205687

Published: 01/01/2025

*Document Version*
Peer-reviewed accepted author manuscript, also known as Final accepted manuscript or Post-print

*Please cite the original version:*
King, A., & Vyatkin, V. (2025). LLM-based Iterative Refinement of Finite-State Machines with STPA Controller Constraints and Generation of IEC 61499 Code. In L. Almeida, M. Indria, M. de Sousa, A. Visioli, M. Ashjaei, & P. Santos (Eds.), *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)* (Proceedings IEEE International Conference on Emerging Technologies and Factory Automation). IEEE. https://doi.org/10.1109/ETFA65518.2025.11205687

This material is protected by copyright and other intellectual property rights, and duplication or sale of all or part of any of the repository collections is not permitted, except that material may be duplicated by you for your research use or educational purposes in electronic or print form. You must obtain permission for any other use. Electronic or print copies may not be offered, whether for sale or otherwise to anyone who is not an authorised user.

# LLM-based Iterative Refinement of Finite-State Machines with STPA Controller Constraints and Generation of IEC 61499 Code

Akira King* and Valeriy Vyatkin*†
*Department of Electrical Engineering and Automation, Aalto University, Espoo, Finland
†Department of Computer Science, Electrical and Space Engineering, Lulea Tekniska Universitet, Sweden
Email: akira.king@aalto.fi, vyatkin@ieee.org

***Abstract*—Large Language Models (LLMs) are increasingly being used in software development and in applications like code generation. While LLMs can provide significant value in the form of time savings in common programming languages like Python, their usability in generating automation software has yet to be studied extensively. In the context of generating control software in the form of IEC 61131-3 compliant code, initial studies suggest LLMs provide a promising avenue for increasing control engineer productivity. However, similar code generation for IEC 61499-based control applications is still scarce. While tools are being developed for this purpose, their capabilities are not yet fully understood, and they often require significant human input to generate the intended outcomes. This paper explores LLM-based code generation for IEC 61499-based applications through iterative prompting. The prompts for the experiments are derived from requirements generated by System-Theoretic Process Analysis (STPA), which provides a systematic approach to creating prompts that also connect to the larger systems engineering workflow. The results indicate that while the approach may be successful in some instances, more work is required to mitigate the issues arising from its application.**

***Index Terms*—STPA, IEC61499, iterative prompting, LLM, finite-state machine**

## I. Introduction

Artificial Intelligence (AI) and Large Language Models (LLMs) have become increasingly popular over the past few years, with common applications ranging from generating images to analyzing data, and programming software. LLMs are capable of generating code in programming languages such as Python, and even Structured Text for IEC 61131-3 based control applications [1] [2] [3]. Research on the use of LLMs in code generation use cases has also indicated promising results [4], with some studies reporting significant time savings [5]. While the use of LLMs in the generation of code is increasing, their use in control automation software, and specifically IEC 61499 is still rather limited. However, steps are being taken towards this direction, as tools are being developed to utilize LLMs in the context of generating IEC 61499 based automation software [6].

While LLM-based code generation in IEC 61499 has become possible and increasingly sophisticated, it still requires significant human input during the generation process to generate functional control software. In fbAssistant, a proof-of-concept tool currently being developed for this exact application [7], this input takes the form of prompts, through which the LLM is guided on the changes it must implement to the Finite State Machine (FSM) being created. Each prompt may outline changes or conditions that the FSM must satisfy. Currently, prompts may introduce unexpected changes in the generated code, and the types of prompts that produce the best results and the most suitable prompting strategies have yet to be identified.

One approach to improving the results generated by LLMs is iterative prompting, with certain iterative prompting strategies demonstrating an increase in truthfulness over the course of 10 iterations [8]. Could similar improvements in results be observed when applying iterative prompting strategies to generating control software with LLMs? Iterating the software with the same prompt or evolving the software with the same prompt could potentially lead to the control software iterations converging to a solution that aligns better with the intended behavior of the control software. Yet, little research has been conducted on this aspect of code generation. If such prompting strategies result in the iterations improving toward a better solution, the approach could be used to generate functional control software that aligns with the intended behavior with little to no human intervention.

However, even if a convergence with certain types of prompts can be observed, a systematic approach to generating such prompts needs to be defined for the potential convergence to be utilized effectively. In this context, System-Theoretic Process Analysis (STPA) could provide a good source of well-defined prompts, and enables the integration of control software generation to systems engineering processes.

This paper explores an iterative prompting method for generating control software in IEC61499-based control applications using STPA controller constraints and an LLM-based FSM generation tool, fbAssistant. Of specific interest are how different types of prompts impact the generated code, as well as the advantages and disadvantages of evolving FSMs with repeated prompts. The rest of the paper is structured as follows. In Section II, the relevant literature is introduced. Section III describes how the experiment was undertaken. In Section IV-B, the results of the experiment are presented and discussed. Finally, in Section V a summary of the findings is presented, and future work is discussed.

## II. Preliminaries

### A. STPA

System-Theoretic Process Analysis (STPA) is a hazard analysis method that provides an attractive starting point for the design and development of new systems [9], as it is able to identify hazards arising from design flaws and the interaction of the components within a system. [10] Central to the STPA method are Control Structures, which are diagrams depicting the relations between controller and controlled processes through control actions and feedback. In a Control Structure, the controllers within a system assert control over controllers and controlled processes lower in the hierarchy through Control Actions (CA) defined in the Control Structure, while system elements lower in the hierarchy may provide feedback to the elements higher in the hierarchy.

The STPA method consists of 4 steps: Determining the purpose of the analysis, Modeling the Control Structure, Identifying Unsafe Control Actions (UCAs), and Identifying Loss Scenarios. For the purpose of the experiment presented in this work, only the first 3 steps are required.

In Step 1 of the analysis, Losses, Hazards and System Constraints are defined, as well as the system boundary. Losses can be summarized as outcomes that are to be avoided, such as the loss of life, damage to equipment or financial loss. Hazards are states of the system that may lead to a Loss, such as an airplane flying too close to terrain, or a nuclear reactor exceeding a temperature limit. System-Constraints are essentially statements that negate the Hazards, e.g. ”The reactor core must not exceed a temperature of 850 degrees Celsius”.

Step 2 of the analysis involves developing a Control Structure diagram. This is often an iterative process, and may be returned to later after identifying flaws or issues in later steps that need to be addressed in design. The Control Structure can be modeled at various levels of abstraction, from simple diagrams with just a few blocks often being used to depict systems that could be modeled at a much more intricate level. In this work, the Control Structure is modeled at a level of abstraction that distinguishes between different actuators and sensors of the pick-and-place machine. This Control Structure is presented in Fig. 1.

Step 3 of the analysis is used to generate UCAs and controller constraints. The former outline how and in which context a Control Action may become unsafe. The ”how” is categorized into four distinct types: provided, not provided, provided too early/too late, provided for too long/too short. For example, a Control Action turning a cooling fan on may be unsafe if provided when the object being cooled is already at its minimum allowed temperature. In this example, the context is that the object is at a minimum temperature.

Controller constraints are derived from the UCAs, and are statements that outline the only kind of behavior allowed by the controller. For example, if a controller can cause a hazard by issuing a certain command in a specific context, the Controller Constraints must be specified such that this command cannot be issued in this context. In this work, rather than outlining each unsafe context in the Controller Constraints, the only safe context is often specified due to being more intuitive and efficient in the case of this specific system.

### B. IEC-61499

The IEC 61499 standard provides a means of creating distributed and intelligent control and automation systems. Central to the standard is the use of Function Block Diagrams which consist of interconnected Function Blocks. These Function Blocks are event-driven and consist of both event and data inputs and outputs. Data-inputs and outputs are always associated with the events they are transmitted with. Within each Function Block is an Execution Control Chart (ECC), a type of finite-state machine that determines the behavior of the Function Block. Each state in the ECC defines the algorithms and output events associated with it. [11]

### C. Pick-and-place machine

The pick-and-place manipulator presented in Fig. 2, used as a simple case study in this paper, is a stationary machine used to transfer items between locations. The machine used in this experiment handles a total of 4 locations where items either appear or disappear at: 3 input trays and 1 output tray. The objective of the pick-and-place machine is to transfer these items, referred to as workpieces, from the input trays to the output tray. The input trays can hold one workpiece each at a time, and the output tray disposes of the workpieces as they are placed in it.

The machine consists of 4 actuators: two horizontal linear actuators, a vertical linear actuator, and a vacuum gripper. The horizontal actuators are used to navigate between the trays, while the vertical actuator is used to lower and raise the vacuum gripper to and from the trays. The output tray is located such that both horizontal actuators must be retracted for the vacuum gripper to be maneuvered above it. The location of input tray 1 corresponds to just the number 1 actuator being fully extended, while the location of input tray 2 corresponds to just the number 2 horizontal actuator being fully extended. Input tray 3 is reached by fully extending both the number 1 and 2 horizontal actuators.

The horizontal actuators are controlled using extend and retract commands to each actuator. The actuators extend/retract while the commands are given. The vertical actuator is controlled with a single command, extend. When TRUE, the vertical actuator is extended, whereas a FALSE value corresponds with the vertical actuator being retracted. The state of the vacuum gripper is toggled using on and off commands.

The pick-and-place machine has inputs corresponding with each tray, indicating whether or not the vacuum gripper is above the tray. Similarly, the pick-and-place machine has inputs indicating if a tray is occupied by an item. There are also two boolean inputs describing whether the vertical actuator is fully extended or fully retracted, and an input describing whether the vacuum is active.

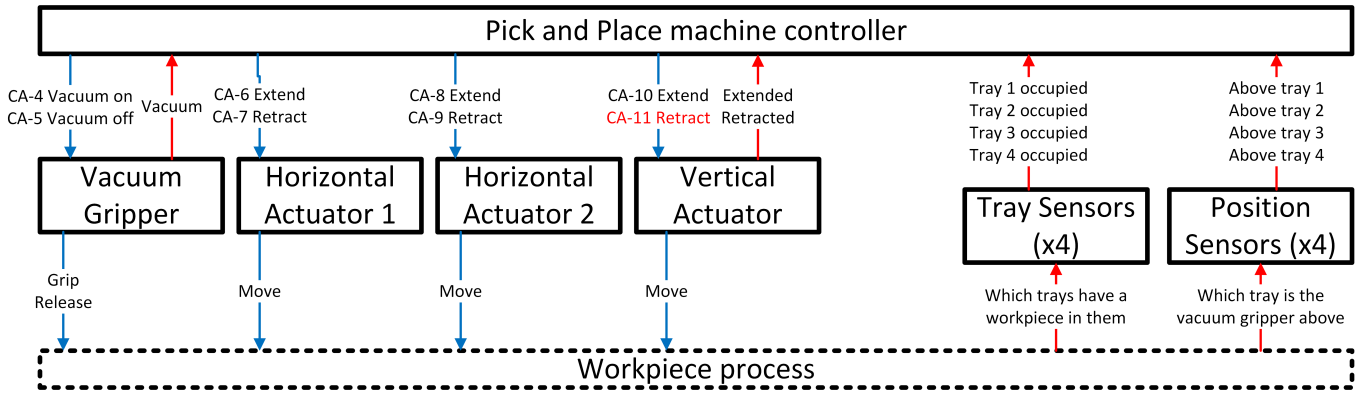


Fig. 1: The Control Structure diagram of the pick-and-place machine. Blue connectors indicate Control Actions, while red connectors indicate feedback. Control Action 11 was mistakenly introduced, but kept in the diagram due to the experiments already having taken place. The diagram omits Control Actions 1, 2, and 3 due to them involving a human operator.

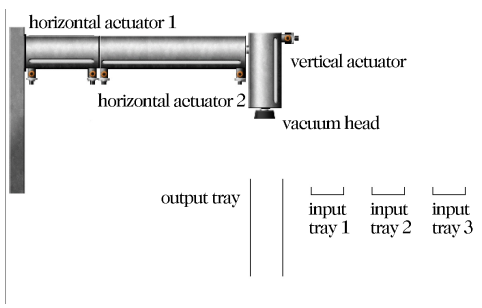


Fig. 2: A visual model of the pick-and-place manipulator used when simulating the system in an IEC 61499 development environment. Workpieces appear at the input trays and are transported to the output tray, where they disappear.

The intended behavior of the pick-and-place machine is described by the following sequence of actions. Once a workpiece arrives at an input tray, the machine maneuvers the vacuum gripper to the input tray, followed by lowering the vacuum gripper and turning the vacuum on. After the vacuum is turned on, the vacuum gripper is raised, and the workpiece is maneuvered to a location above the output tray. Then the workpiece is lowered to the output tray, and the vacuum is turned off. The vacuum gripper is then raised. This sequence is repeated for any workpieces appearing in the input trays, and all workpieces must eventually reach the output tray.

### D. fbAssistant

The Function Block Assistant is a proof-of-concept tool developed by Flexbridge AB, which follows an iterative methodology to develop automation logic with LLM support. The workflow of the tool is presented in Fig. 3. In the current work, the tool is mainly used as an interface for OpenAI's GPT models and to provide a visual means of checking the code generated by the LLM. The tool also allows for a very straightforward process of converting the LLM's output to executable IEC 61499 code on both soft Programmable Logic Controllers (PLCs) for simulation and real PLCs.

The process begins with the user providing natural-language descriptions of the required control logic. These descriptions are then processed by an AI model, which generates a corresponding state machine representation. Once the state machine is created, it is visualized in the tool for user verification. This visualization helps identify any missing transitions or incorrect states. If adjustments are needed, the user refines the textual input and submits an updated request to the AI.

After validation, the state machine is converted into a function block that is compatible with industrial automation tools such as EcoStruxure Automation Expert. The generated function block is then deployed to a virtual or physical environment for testing. Testing involves simulating the generated control logic within a virtual commissioning environment. This allows engineers to verify the expected behavior before deploying it to the actual hardware. The refinement process ensures that any discrepancies between expected and actual behavior are resolved prior to full implementation.

Fig. 4 illustrates one step in the state machine refinement process supported by fbAssistant. Given the incomplete state machine of the pick-and-place controller on the left, the user provided five comments in natural language, each of which was correctly addressed by the assistant, making the corresponding modifications to the state machine.

## III. Methods

In order to assess the approach explored in this work, an experiment was conducted with the function block of a pick-and-place machine in IEC61499. First, an STPA analysis of the system was performed based on information about the available actuators, sensors, and intended behavior of the pick-and-place machine. Only the first 3 steps of the STPA analysis were completed, as the Loss Scenarios derived in the fourth step of the STPA analysis were not used in this experiment.

The results of the STPA analysis include the Losses and Hazards presented in Table I, the Control Structure diagram

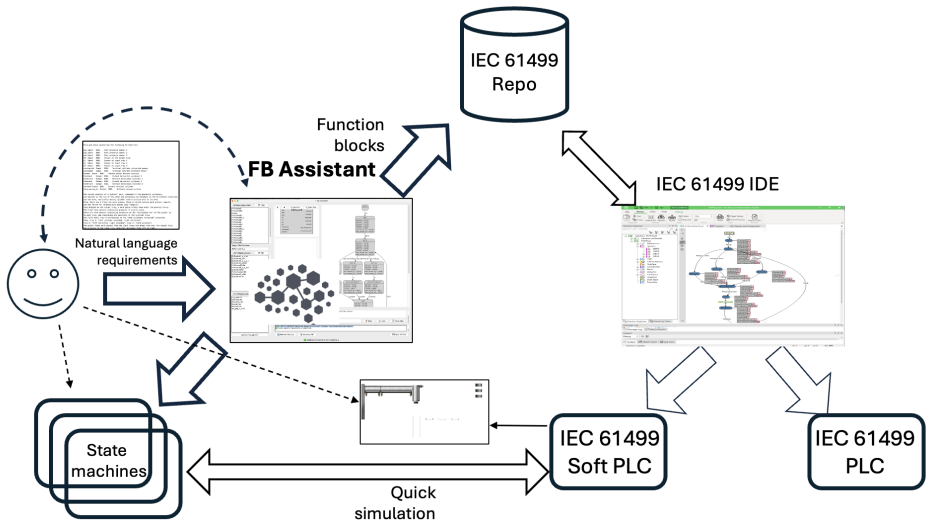


Fig. 3: Overview of the fbAssistant workflow, showing the step-by-step process from natural language requirements to executable control logic.

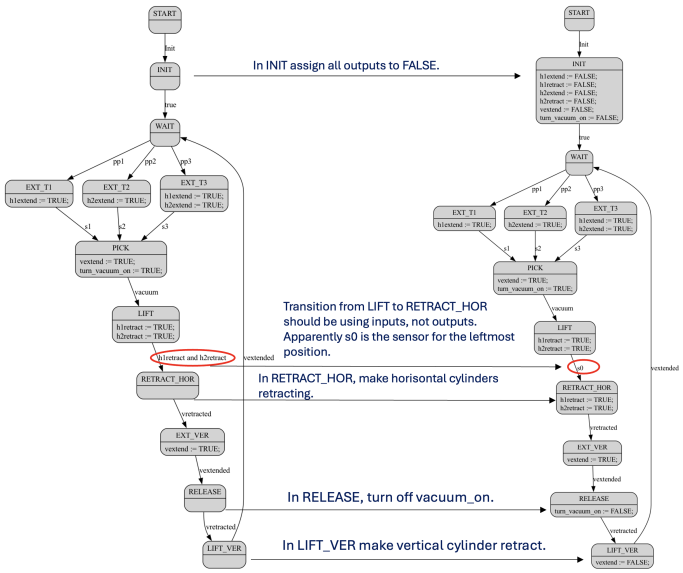


Fig. 4: Refinement of the state machine for the pick-and-place manipulator.

TABLE I: Losses and Hazards

| ID | Description | Associated result |
|---|---|---|
| L-1 | Injury or loss of life | H-1, H-2, H-3 |
| L-2 | Damage to property | H-1, H-2, H-3 |
| L-3 | Loss of mission (unable to continue picking and placing) | H-1, H-2, H-3, H-4 |
| H-1 | A workpiece is in freefall. | L-1, L-2, L-3 |
| H-2 | The pick and place machine collides with an obstacle. | L-1, L-2, L-3 |
| H-3 | A workpiece is placed in neither of the 4 trays. | L-1, L-2, L-3 |
| H-4 | All workpieces are not transferred from an input tray to the output tray. | L-3 |

presented in Fig. 1, and a total of 33 Unsafe Control Actions, some of which are presented in Table II. Lastly, the results also include the controller constraints presented in Table III. Unfortunately, Control Action 11 was later determined slightly inaccurate, as no explicit ”retract” command is sent to the vertical actuator, and retraction is rather accomplished through the absence of an ”extend” command. However, the controller constraints relating to UCAs derived from the retract command were adjusted, and the experiments were conducted with the adjusted controller constraints: C-8, C-9, and C-10.

Following the STPA analysis, the control software within the controller’s function block was drafted as a finite-state machine through natural language prompts in the fbAssistant tool. This initial finite-state machine is presented in Fig. 5. Once an approximate structure for the finite-state machine was achieved, the finite-state machine was iterated using 9 different prompts and 20 iterations for each prompt, resulting in a total of 180 iterations. Each prompt was applied to the result of the previous iteration in a recursive manner, instead of the original finite-state machine. These prompts were based on the controller constraints derived in the STPA analysis, and were of the format ”If the following constraints do not hold for the state machine, modify it such that the constraints hold:” followed by the controller constraint. During the first round of iterations, prompts included only one controller constraint each. These controller constraints are documented in Table III.

The changes made during each iteration were documented and tracked according to a subjective evaluation of whether the changes made in the iteration were positive, negative, or neutral. Examples of positive changes are those that fix an issue, such as dropping the workpiece or incorrect syntax, while negative changes are those that introduce unwanted behavior or incorrect syntax. Neutral changes often included

TABLE II: 4 of 33 Unsafe Control Actions

| ID | Description | Associated Hazard |
|---|---|---|
| UCA-5-2-1 | PnP Controller provides the Vacuum Gripper with the vacuum off command when the vacuum head is not in the lowered position above the output tray. | H-1, H-3, H-4 |
| UCA-6-1-1 | PnP Controller does not provide the H1 Actuator with the extend command when maneuvering to the 1st or 3rd input tray, resulting in the Vacuum Head being positioned above the wrong tray. | H-4 |
| UCA-10-1-1 | PnP Controller does not provide the V Actuator with the extend command during the workpiece picking sequence, resulting in the workpiece not being picked by the Vacuum Head. | H-4 |
| UCA-10-1-2 | PnP Controller does not provide the V Actuator with the extend command during the workpiece placing sequence, resulting in the workpiece not being placed or being dropped. | H-1, H-3, H-4 |

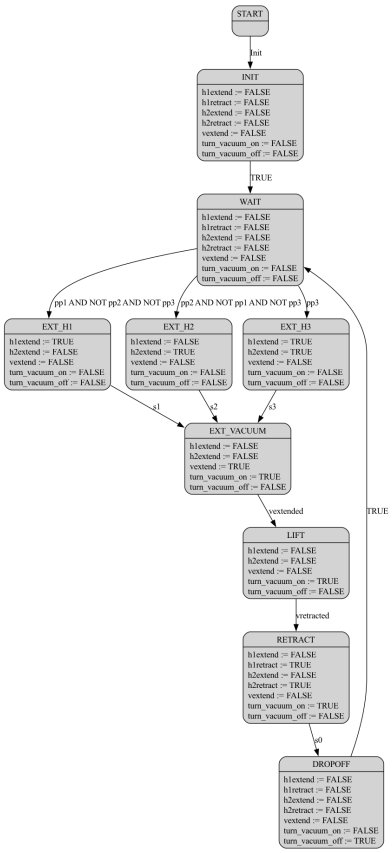


Fig. 5: The initial finite-state machine, which was used as the starting point for each each set of 20 iterations.

unnecessarily defining the state of a variable, or adding a condition that likely would not change the behavior of the system.

During the initial round of over 180 iterations, some intermediate hypotheses were made regarding factors that affect the success of prompting with the controller constraints:

TABLE III: Controller Constraints

| | |
|---|---|
| C-1 | The Controller must provide the vacuum on command when and only when the vertical actuator is in the extended position and above an input tray. |
| C-2 | The Controller must provide the vacuum off command when and only when the vertical actuator is in the extended position and above a vacant tray. |
| C-3 | The Controller must provide the extend command to the number 1 horizontal actuator when and only when maneuvering to the 1st or 3rd input trays. |
| C-4 | The Controller must provide the the retract command to the number 1 horizontal actuator when and only when maneuvering to the output tray. |
| C-5 | The Controller must provide the extend command to the number 2 horizontal actuator when and only when maneuvering to the 2nd and 3rd input trays. |
| C-6 | The Controller must provide the retract command to the 2nd horizontal actuator when and only when maneuvering to the output tray. |
| C-7 | The Controller must provide the vextend command when and only when it is in the picking or placing sequences. |
| C-8 | The Controller must stop providing the vextend command to the vertical actuator before performing any horizontal movement. |
| C-9 | The Controller must stop providing the vextend command in the picking sequence after and only after the vacuum has been turned on, and the vertical actuator has fully extended. |
| C-10 | The Controller must stop providing the vextend command in the placing sequence after and only after the vacuum has been turned off, and the vertical actuator has fully extended. |
| C-11 | The Controller must not provide commands to the horizontal actuators if the vertical actuator is not fully retracted. |

- Using language that is inconsistent with the variables used in the existing control software may degrade the resulting code.
- Each prompt should aim to fully describe the behavior of the actuator or device being controlled, in order to limit poor assumptions made by the LLM.

These hypotheses were tested by prompting with controller constraints C-1 and C-2 concurrently. When prompted separately these controller constraints introduced problems that were identified as potentially being preventable had the other constraint been prompted at the same time. These constraints were also rewritten such that the conditions were altered: ”above an input tray” and ”above a vacant tray” were expressed with input variables s0, s1, s2, and s3 instead.

## IV. Results

### A. Initial round of iterations

The initial round of iterations produced varying results, with some prompts generating encouraging results, while most prompts led to dissatisfactory outcomes. Fig. 6 presents a summary of these results in the form of neutral, negative, and positive changes introduced by the LLM in each iteration for each prompt.

The number of overall changes introduced in each set of iterations and the maximum number of changes introduced at once vary significantly from one constraint to another, with some controller constraints generating a maximum of 3 changes in one iteration, while others introduced up to 12

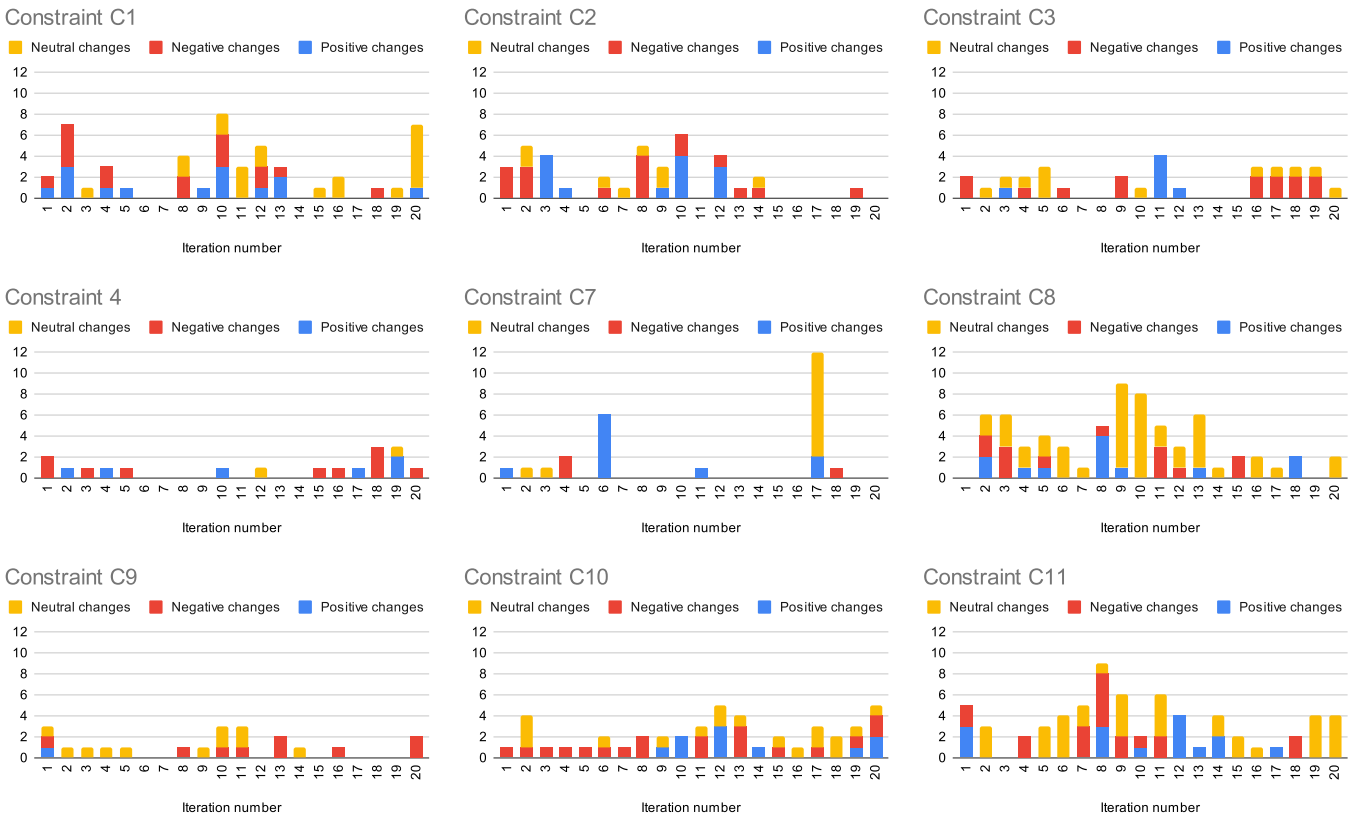


Fig. 6: The number and types of changes per iteration for each of the prompts for controller constraints C1, C2, C3, C4, C7, C8, C9, C10, and C11. The vertical axes represent the number of changes and the horizontal axes the iteration during which the changes occured.

changes in one iteration. Over the course of 20 iterations, some controller constraints generated roughly 20 changes, while others generated up to 69 changes.

Similarly, while some controller constraints resulted in a significant amount of neutral changes (i.e. benign with little to no impact on the behavior of the code), others were quick to introduce negative changes. However, the numbers of positive and negative changes do not communicate the severity of the changes, and therefore the numbers of changes do not directly reflect a successful or unsuccessful outcome. The absolute number of changes also does not capture the number of types of changes introduced. For example, some iterations introduced up to 12 changes which could be all classified as a single type of change, while other iterations would introduce several different types of changes at once.

In many instances, positive changes introduced in later iterations were considered positive due to addressing or reversing negative changes made in earlier iterations. For example, in one instance, the value of an output variable in a specific state was changed back and forth between TRUE and FALSE for several iterations.

The only set of iterations that could be considered truly successful was the set that used controller constraint C-7 as the basis for its prompts. The iterations in this set introduced a total of 10 positive changes, 3 negative changes, and 12 neutral changes. The resulting finite-state machine, presented in Fig. 7, ensures that the vertical actuator is fully extended before releasing the workpiece. This is unlike the original finite-state machine, which dropped the workpiece without the vertical actuator being extended at all. The best-behaved versions of the finite-state machine with controller constraint C-7 were reached in iteration 11. This version also ensured that the vertical actuator was fully retracted before proceeding to the ”WAIT” state, essentially preventing the vertical actuator from being in the extended state when maneuvering to one of the input trays. The LLM was also hesitant to introduce changes to this version of the finite-state machine, with no changes being introduced for the following 5 iterations.

Other sets of iterations failed due to a variety of reasons. One of the most common and significant negative changes introduced through the prompts was the removal of the states describing how to maneuver to each input tray. These changes often occurred over the course of several iterations, starting from small changes with one variable, then escalating to the removal of transitions, and finally the removal of a state. In many cases, the state describing how to reach the 3rd input tray would first be altered to mimic the state describing how to reach the 2nd input tray, and then eventually be removed

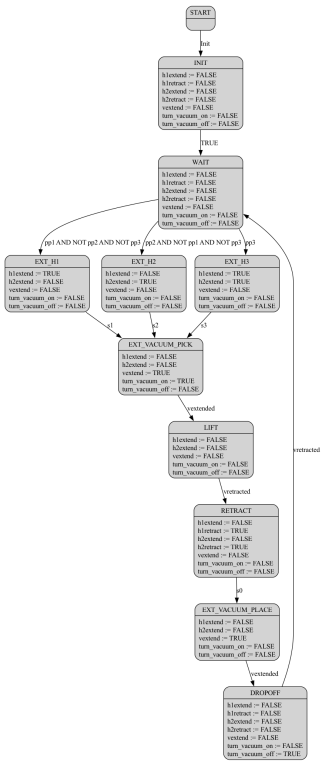


Fig. 7: The finite-state machine from iteration 16 with controller constraint C-7. The iteration following the one presented in the figure introduced mostly superfluous variable assignments, but also one assignment that ensures that the pick and place machine is truly stopped during the ”EXT_VACUUM_PLACE” state.

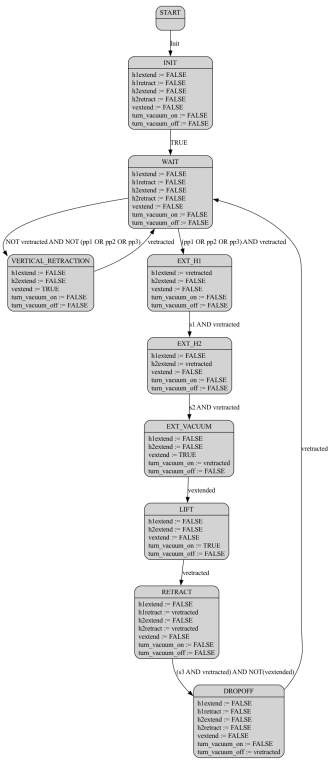


Fig. 8: The finite-state machine from iteration 20 with controller constraint C-11. In previous iterations the states ”EXT_H1” and ”EXT_H2” were entirely disconnected, but they were reconnected again in a new order presented above.

entirely. These iterations could perhaps be best described as the finite-state machine migrating or drifting. Through migration, the FSM can evolve into an entirely different structure over the course of just a few iterations.

Interestingly, in some cases states would be reintroduced after being disconnected from the rest of the finite-state machine for several iterations. For example, in Fig. 8 the LLM has reintroduced states ”EXT_H1” and ”EXT_H2” to the finite-state machine, albeit in a different pattern to the original finite-state machine.

Another common negative change introduced by the LLM was the introduction of new variables, for example, controller constraints C-1 and C-2 both introduced new input variables ”aboveTray” and ”vacant_tray”, which correspond with the context provided in the constraints. These should likely be defined through the variables already implemented in the finite-state machine, such as the sensor inputs s0, s1, s2, and s3. Interestingly, in the case of controller constraint C-9, the LLM introduced a new output variable ”vextend_stop”, which, after first being introduced to states such as ”EXT_VACUUM”, was also later correctly introduced to the initialization state. This was not explicitly requested by the user.

In many of the iterations, the LLM would assume the state of similar or complementary variables even when the controller constraint only defined the behavior of a single variable. For example, the LLM would often assume that if the ”turn_vacuum_on” command was to be provided in a specific situation, the ”turn_vacuum_off” command should be provided in all other situations. However, the correct behavior of the finite-state machine includes many states in which neither of the commands should be provided.

Some of the iterations introduced transition conditions that depended on the value of an output variable instead of an input variable. Examples include states in which the vacuum is commanded to turn on, and the following transition condition referring to the output variable ”turn_vacuum_on” instead of the input variable ”vacuum”. These transition conditions effectively check nothing, while the correct transition condition would check the state of the vacuum.

Some iterations introduced changes to aspects of the finite-state machines that did not directly correlate with information provided in the controller constraint. For example, in the set of iterations with controller constraint C-8, the LLM changed the value of the ”turn_vacuum_off” command several times, despite a lack of any reference to the behavior of the vacuum gripper in the controller constraint itself.

In four instances, the LLM returned an unparseable JSON file. In these cases the iteration was run a second time which led to a parseable output in all four instances. Similarly, the LLM produced outputs with syntactical errors in a few in-

stances, including missing spaces or incorrect notation. These errors never persisted for more than 3 iterations.

### B. Second round of iterations

In addition to the 180 iterations conducted in 9 sets of 20 iterations for each prompt separately, 1 set of 10 iterations was conducted with a prompt including both controller constraints C-1 and C-2. The number of changes introduced in this set of iterations was similar to the earlier sets despite the set spanning only 10 iterations, with 19 positive, 20 negative, and 6 neutral changes over the course of 10 iterations.

Combining the two controller constraints into one prompt appears to improve the results. Most notably, new variables were not introduced, and the iterations did not drift towards removing the "EXT_H3" state unlike iterations conducted with each of the controller constraints C-1 and C-2 separately.

## V. Conclusions

This paper has described and presented results of the first attempts in evolving finite-state machines with the aid of an LLM and prompts based on controller constraints derived from an STPA analysis. While the results shed light on the characteristics that may be desirable in prompts used to iterate control software, the prompting method was only applied successfully in 1 of 10 prompts tested in this paper. Key findings are summarized below:

- As implemented in this paper, iteratively evolving the controller with controller constraints often falls short of the intended outcome.
- The results of each iteration tend to drift/migrate slowly over the course of several iterations, however, not always in a positive direction.
- Prompts that define the intended behavior of a specific actuator poorly or partially appear to leave more room for the LLM to assume behavior and introduce unwanted changes into the results. A more complete description of the intended behavior of the actuator in each prompt appears to improve results and mitigate some of the identified issues.
- Controller constraints should likely be formulated such that they describe the context in terms of the variables existing in the FSM, in order to limit the generation of superfluous variables.
- Prompts in this paper were formatted so as to not undermine the "confidence" of the LLM, as such prompts have been identified as detrimental to the outcomes of iterative prompting [8]. Further improving the language used in the prompt may reduce the number of negative and superfluous changes made by the LLM.

A key weakness of the approach explored in this paper is its susceptibility to introducing unwanted drift to the generated code over the course of several iterations. This may lead to the correct parts of the existing code to be altered significantly. These issues could potentially be addressed by improving prompt formulation and limiting the number of iterations. Using several controller constraints together or formulating them such that they each describe the behavior of the component being controlled more completely could likely also generate significant improvements. However, more work is required to confirm this.

In addition to confirming the impact of combining controller constraints and formulating constraints that fully describe behavior of each component, future work should focus on exploring alternative sources for prompts, for example system descriptions, and how they could be introduced into iterations. Additionally, future work should identify the stages of code generation in which iterating the code with requirements provides the best results, as the finite-state machine used as the starting point for each set of iterations was already in a rather mature stage compared to the true starting point of an empty FSM.

While the iterative prompting method presented here provided interesting insight into the behavior of LLMs in the context of evolving FSMs with an iterative prompting approach, alternative iterative prompting methods should also be explored, such as those iterating on the original code compared to iterating recursively with the previous iteration of code.

## VI. Acknowledgements

This work was supported, in part, by the SEAMLES project, funded by the nuclear safety SAFER program of Finland, and Horison Europe project MEDUSA, which is focussing on the development of manufacturing as a service platform supported by AI.

## References

[1] H. Koziolek, S. Gruener, and V. Ashiwal, "ChatGPT for PLC/DCS control logic generation," in *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–8, 2023.

[2] H. Koziolek and A. Koziolek, "LLM-based control code generation using image recognition," in *Proceedings of the 1st International Workshop on Large Language Models for Code*, LLM4Code '24, (New York, NY, USA), p. 38–45, Association for Computing Machinery, 2024.

[3] K. Tran, J. Zhang, J. Pfeiffer, A. Wortmann, and B. Wiesmayr, "Generating PLC code with universal large language models," in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–8, 2024.

[4] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," 2024.

[5] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The impact of AI on developer productivity: Evidence from GitHub Copilot," 2023.

[6] P. Ovsiannikova, T. Liakh, P. Jhunjhunwala, and V. Vyatkin, "Generative AI Co-Pilot for rapid prototyping of IEC 61499 control applications," in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 1–4, 2024.

[7] V. Vyatkin, S. Patil, D. Drozdov, and A. Shalyto, "LLM-based iterative requirements refinement in FSM with IEC 61499 code generation," in *Proceedings of the 2025 IEEE International Conference on Industrial Informatics (INDIN)*, IEEE, 2025. accepted.

[8] S. Krishna, C. Agarwal, and H. Lakkaraju, "Understanding the effects of iterative prompting on truthfulness," 2024.

[9] N. G. Leveson, "Safety analysis in early concept development and requirements generation," *INCOSE International Symposium*, vol. 28, no. 1, pp. 441–455, 2018.

[10] N. G. Leveson and J. P. Thomas, "STPA handbook," *Cambridge, MA, USA*, 2018.

[11] J. H. Christensen, T. Strasser, A. Valentini, V. Vyatkin, and A. Zoitl, "The IEC 61499 function block standard: Overview of the second edition," *ISA autom. Week*, vol. 6, pp. 6–7, 2012.